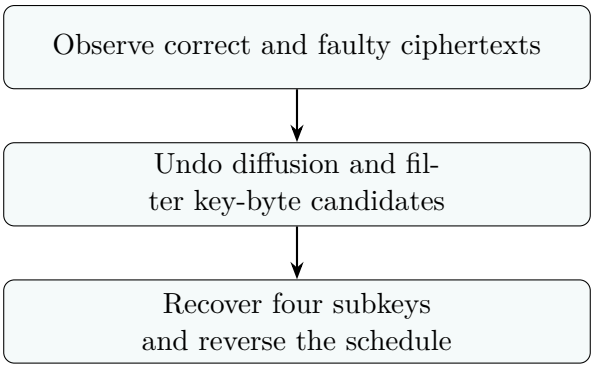
Polytech Montpellier — PFE 5A

Understanding Every Function in

`piret_sm4.py`

A practical, step-by-step reading guide



Guilherme Martins Specht
Industrial collaboration: STMicroelectronics

Independent guide — November 13, 2026

Contents

1 How to read the complete file 1

2 `sm4_fault_encrypt()`: simulate the target 2

3 `inverse_rows()`: prepare inverse diffusion 2

4 `inverse_linear(value)`: apply the inverse 8

5 Visual walkthrough: follow the loops through the matrix 9

6 `sm4_byte_candidates()`: solve one observation 14

7 `reverse_sm4()`: recover the master key 15

8 `peel_sm4()`: expose an earlier round 15

9 `recover_sm4_master_key()`: coordinate the attack 16

10 `main()`: connect the terminal interface 17

11 Follow one real execution from the command to the recovered key 19

12 One complete practical story 26

1 How to read the complete file

This guide covers the imports, all eight top-level functions, the nested fault callback, and the terminal entry point. It explains the purpose of each code block, important operators and practical values. Repeated iterations are summarized rather than printing 256 guesses or an entire matrix elimination. The examples are checked by executing the current implementation when this PDF is built. The source itself is not modified.

Scope. The name "Piret SM4" identifies this project's controlled known-byte DFA model. It is not an unchanged transfer of the historical AES attack. Exact fault round, byte and XOR magnitude are simulation assumptions. No physical STM32 key recovery is established by these Python examples.

1.1 The three roles that must remain separate

Role	What it knows and does
Simulated target	Knows plaintext and secret key; produces clean or faulted encryption.
Attacker / solver	Receives ciphertexts and permitted fault descriptors; filters candidates and recovers keys.
Teaching interface	Displays intermediate values to explain the simulation; these values must not become secret inputs to the solver.

1.2 Imports and representation

`argparse` handles command-line options. `lru_cache` stores the fixed inverse matrix. `struct` packs and unpacks four unsigned 32-bit words. `aes` supplies the shared block-length validator; this does not make the attack an AES attack. `sm4` supplies the public cipher, S-box, linear maps and constants.

The format `">4I"` means four unsigned 32-bit integers in big-endian order. Byte 0 is the most significant byte of a word. Bit index 0 in the binary matrix is the least significant bit. The following operators have distinct roles:

Python	Meaning here
`a ^ b`	XOR; addition/subtraction over GF(2)
`a & b`	Select bits; for sets, intersection
`a | b`	Assemble bits in an integer
`a << n`, `a >> n`	Shift left/right by n bits
`a % 4`	Wrap an integer index into 0, 1, 2, 3
`items[::-1]`	Reverse a sequence

1.3 Running the example

```
python src/piret_sm4.py --step
python src/debug_piret_sm4.py
```

The first command presents the attack in the terminal. The second opens the source-and-variables debugger with substituted numerical equations.

2 `sm4_fault_encrypt()`: simulate the target

2.1 Inputs and return values

`plaintext` and `key` are 16-byte values. `byte` is the affected S-box byte, from 0 to 3; `None` requests normal encryption. `mask` is a nonzero byte XOR error. `fault_round` is zero-based: 31 is the last of 32 rounds. The return value is `(ciphertext, trace)`. Only the ciphertext is returned through the oracle to the recovery engine.

2.2 Validate the requested fault

When a byte is selected, the first condition checks the byte position and mask range 1–255. The next condition requires a round in 0–31. Invalid descriptors raise `ValueError`; a missing fault byte disables injection.

2.3 Prepare the teacher's trace

`sm4.key_expansion(key)` creates the target's round keys. The loop builds one trace dictionary per round, initially containing its index and round key. The `fields` dictionary translates event names into compact teaching labels: S-box input a, S-box output b, linear output and new state word. These keys and internal values belong to the target/teacher view, not the solver.

2.4 Nested `observer()`: the exact fault instruction

The cipher calls this callback at internal events. Its condition requires all three properties: selected round, event `"Sbox input"`, and a selected byte.

```
value ^= mask << (24 - 8 * byte)
```

For byte 0 and mask 01, the shift is $24 - 8(0) = 24$:

$$\texttt{01} \ll 24 = \texttt{01000000}, \qquad \texttt{96FF9D45} \oplus \texttt{01000000} = \texttt{97FF9D45}.$$

For byte 2 the shift is 8, so the same mask becomes `00000100`. The selected byte is corrupted before nonlinear substitution; XOR does not mean replacing the entire byte with the mask.

If the event name is in `fields`, the callback saves the value in that round's trace. It returns the value, possibly changed. The real `sm4.py` encryption uses that returned value, which is what makes the intervention effective. The final call passes this observer to `sm4_encrypt_block`.

A physical glitch changes operating conditions rather than executing this XOR statement. A one-bit mask in Python is exact; a pulse on the board may cause no error, multiple errors, an instruction fault or a reset.

3 `inverse_rows()`: prepare inverse diffusion

3.1 Why this function exists

The data linear map is

$$L(x) = x \oplus \mathrm{ROL}_2(x) \oplus \mathrm{ROL}_{10}(x) \oplus \mathrm{ROL}_{18}(x) \oplus \mathrm{ROL}_{24}(x).$$

It spreads the S-box output difference across a 32-bit word. The solver needs the inverse map to restore that difference before guessing a key byte. Linearity over GF(2) lets us write $y = Mx$ with a binary 32×32 matrix. This function calculates M^{-1}, not a recovered secret.

3.2 Cache the fixed matrix

```
@lru_cache(maxsize=1)
```

Because the matrix does not depend on the key or input, repeated calls reuse its cached result. This lasts within the process until the cache is cleared or the process ends; the comment "first call, ever" should be read with that limit.

3.3 Step 1: discover each column

```
basis_word = 1 << bit_index
transformed_word = sm4.linear(basis_word)
columns.append(transformed_word)
```

The loop visits 32 unit vectors: `00000001`, `00000002`, `00000004`, and so on. Applying L to unit vector i gives column i of M. For the first column:

$$L(\texttt{00000001}) = \texttt{00000001} \oplus \texttt{00000004} \oplus \texttt{00000400} \oplus \texttt{00040000} \oplus \texttt{01000000} = \texttt{01040405}.$$

This column says which output bits depend on input bit 0.

3.4 Step 2: collect rows and attach the identity

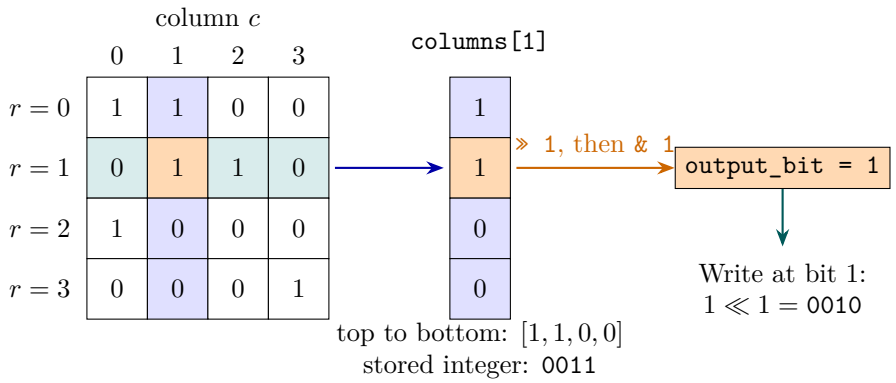
Goal: take the already known matrix columns, reconstruct each row, and attach an identity row. This step does not invert anything yet. The result is the starting augmented matrix for the next step.

```
rows = []
for row_index in range(32):
    matrix_row = 0
    for column_index in range(32):
        output_bit = (columns[column_index] >> row_index) & 1
        matrix_row |= output_bit << column_index
    identity_bit = 1 << (32 + row_index)
    augmented_row = matrix_row | identity_bit
    rows.append(augmented_row)
```

Indentation is essential: the identity is attached *after* all columns of one row have been read. The function appends one packed integer per outer-loop iteration, not one integer per cell.

A drawing at the exact point where the bit is read

Use a four-bit teaching matrix, with stored columns $[\texttt{0101}, \texttt{0011}, \texttt{0010}, \texttt{1000}]$ in binary. The highlighted cell is row 1, column 1. Matrix rows run downward; matrix columns run rightward. Printed binary numbers put bit 0 on the right.



The column is not moved. We read one coefficient from its packed integer and write that coefficient into the packed row being constructed.

Every line: substituted values, result, and purpose

For the small example only, use width 4 instead of 32 in the two ranges and in the identity shift. The next worked iteration has $r = 1$, $c = 1$. The preceding column $c = 0$ contributed 0, so the accumulated row is still `0000`. Binary words below are padded to four or eight positions; loop indices and shift counts are decimal.

1. `rows = [].` Create an empty list of augmented rows. It will eventually contain four integers in this example, or 32 in SM4. The list is initialized once, before either loop. It must retain earlier rows when a new row begins.

2. `for row_index in range(4):.` The outer iterator produces $0, 1, 2, 3$. We are showing its second iteration, so `row_index = 1`. This selects the matrix row to reconstruct; it stays fixed throughout the inner loop. Row 0 has already been appended.

3. `matrix_row = 0.` At the beginning of row 1, set the accumulator to `0000`. This removes the bits used to construct the previous row; reusing that row would mix unrelated coefficients. This line executes once per row, not once per column.

4. `for column_index in range(4):.` Visit $c = 0, 1, 2, 3$ within fixed row $r = 1$. At $c = 1$, read the second stored column: `columns[1] = 0011`. The position in the list selects a column; the row index selects a bit within that column.

5. `output_bit = (columns[column_index] » row_index) & 1.` Substitute each operand:

$$\begin{aligned}\texttt{output_bit} &= (\texttt{0011} \gg 1) \,\&\, \texttt{0001} \\ &= \texttt{0001} \,\&\, \texttt{0001} = 1.\end{aligned}$$

Why shift? To bring the coefficient for row 1 into bit position 0. Why AND with 1? To discard all other coefficients and obtain exactly 0 or 1. For contrast, column 0 gives $(\texttt{0101} \gg 1) \,\&\, \texttt{0001} = \texttt{0010} \,\&\, \texttt{0001} = 0$. The extra 1 in position 1 is irrelevant to this selected matrix entry.

6. `matrix_row |= output_bit « column_index.` The augmented assignment is shorthand for `matrix_row = matrix_row | (output_bit « column_index)`. For $c = 1$ the complete update is

$$\texttt{matrix_row} = \texttt{0000} \,|\, (1 \ll 1) = \texttt{0000} \,|\, \texttt{0010} = \texttt{0010}.$$

Why shift left? In the reconstructed row, column c is stored at bit position c. Why OR? Set that position while preserving all previously assembled bits. In the next iteration $c = 2$, another extracted 1 contributes `0100`:

$$\texttt{0010} \,|\, (1 \ll 2) = \texttt{0010} \,|\, \texttt{0100} = \texttt{0110}.$$

Column 3 contributes 0, so the completed row remains `0110`.

7. `identity_bit = 1 « (4 + row_index).` This line executes after the inner loop. Use $r = 1$:

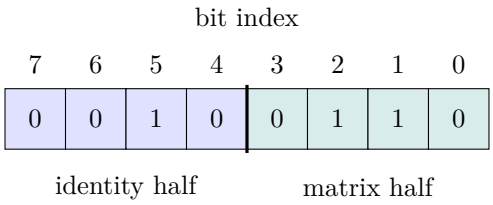
$$\texttt{identity_bit} = 1 \ll (4 + 1) = 1 \ll 5 = \texttt{00100000}.$$

Why add 4? The low four positions already hold the matrix row. The identity must occupy the next four positions. Why add r? Row r of the identity has its 1 in column r. In SM4 this expression uses $32 + r$, not $4 + r$.

8. `augmented_row = matrix_row | identity_bit`. Pad both operands to eight bits before combining them:

$$\begin{array}{rcl} \text{matrix row} & = & \texttt{0000 0110} \\ \text{identity row} & = & \texttt{0010 0000} \\ \hline \text{bitwise OR} & = & \texttt{0010 0110}. \end{array}$$

The result is decimal 38, hexadecimal `26`. OR does not mean concatenating decimal numbers: the two integers occupy non-overlapping bit positions.



Mathematics writes $[M \mid I]$, but a printed integer lists its high half first, so storage appears as $[I \mid M]$. The row operations will act on both halves.

9. `rows.append(augmented_row)`. Row 0 previously produced `00010011` (hex `13`). Append row 1's `00100110` (hex `26`):

$$\texttt{rows} = [\texttt{13}] \quad \longrightarrow \quad [\texttt{13}, \texttt{26}] \qquad \text{(hexadecimal entries)}.$$

Then the outer loop advances to row 2, resets the accumulator, and repeats. This append is why completed rows are not lost when `matrix_row` is reset.

The complete four-bit run: every iteration, no guessed values

In each table, C is the stored column; "shifted" is $C \gg r$; $b = (C \gg r)\ \&\ 1$; "placed" is $b \ll c$; and "row before/after" shows the OR accumulator. Padded digit strings are binary; the row and column indices, shift counts and decimal shift results are ordinary decimal integers.

Outer iteration: row_index = 0. Start with `matrix_row = 0000`.

c	C	Shifted	b	Placed	Row before	Row after
0	0101	0101	1	0001	0000	0001
1	0011	0011	1	0010	0001	0011
2	0010	0010	0	0000	0011	0011
3	1000	1000	0	0000	0011	0011

$$\texttt{identity_bit} = 1 \ll (4 + 0) = 16 = \texttt{00010000},$$
$$\texttt{augmented_row} = \texttt{00000011} \mid \texttt{00010000} = \texttt{00010011}.$$

Append to `rows`. List so far (hex): `13`.

Outer iteration: row_index = 1. Start with `matrix_row = 0000`.

c	C	Shifted	b	Placed	Row before	Row after
0	0101	0010	0	0000	0000	0000
1	0011	0001	1	0010	0000	0010
2	0010	0001	1	0100	0010	0110
3	1000	0100	0	0000	0110	0110

$$\texttt{identity_bit} = 1 \ll (4 + 1) = 32 = \texttt{00100000},$$
$$\texttt{augmented_row} = \texttt{00000110} \mid \texttt{00100000} = \texttt{00100110}.$$

Append to `rows`. List so far (hex): `13, 26`.

Outer iteration: row_index = 2. Start with `matrix_row = 0000`.

c	C	Shifted	b	Placed	Row before	Row after
0	0101	0001	1	0001	0000	0001
1	0011	0000	0	0000	0001	0001
2	0010	0000	0	0000	0001	0001
3	1000	0010	0	0000	0001	0001

$$\texttt{identity_bit} = 1 \ll (4+2) = 64 = \texttt{01000000},$$
$$\texttt{augmented_row} = \texttt{00000001} \,|\, \texttt{01000000} = \texttt{01000001}.$$

Append to `rows`. List so far (hex): `13, 26, 41`.

Outer iteration: row_index = 3. Start with `matrix_row = 0000`.

c	C	Shifted	b	Placed	Row before	Row after
0	0101	0000	0	0000	0000	0000
1	0011	0000	0	0000	0000	0000
2	0010	0000	0	0000	0000	0000
3	1000	0001	1	1000	0000	1000

$$\texttt{identity_bit} = 1 \ll (4+3) = 128 = \texttt{10000000},$$
$$\texttt{augmented_row} = \texttt{00001000} \,|\, \texttt{10000000} = \texttt{10001000}.$$

Append to `rows`. List so far (hex): `13, 26, 41, 88`.

The loop terminates with four packed rows. Only now does Gaussian elimination begin. No ciphertext, plaintext or secret key was needed to construct these public matrix rows.

Back to real SM4: the exact same statements with width 32

For `row_index = 1` and `column_index = 1`, the real column is $L(2^1) = \texttt{0208080A}$. The previous column gave a zero coefficient in this row, so `matrix_row = 0`. Substitute the real values:

$$\begin{aligned}\texttt{output_bit} &= (\texttt{0208080A} \gg 1) \,\&\, 1 \\ &= \texttt{01040405} \,\&\, 1 = 1, \\ \texttt{matrix_row} &= \texttt{00000000} \,|\, (1 \ll 1) = \texttt{00000002}.\end{aligned}$$

The following table completes this real row across all 32 columns. Hexadecimal is used for 32-bit words to avoid wrapping long binary strings; b is a single bit. Every cell extraction and accumulation is included for this one row.

c	$L(2^c)$	Shifted by 1	b	Placed	Row before	Row after
0	01040405	00820202	0	00000000	00000000	00000000
1	0208080A	01040405	1	00000002	00000000	00000002
2	04101014	0208080A	0	00000000	00000002	00000002
3	08202028	04101014	0	00000000	00000002	00000002
4	10404050	08202028	0	00000000	00000002	00000002
5	208080A0	10404050	0	00000000	00000002	00000002
6	41010140	208080A0	0	00000000	00000002	00000002
7	82020280	41010140	0	00000000	00000002	00000002
8	04040501	02020280	0	00000000	00000002	00000002
9	08080A02	04040501	1	00000200	00000002	00000202
10	10101404	08080A02	0	00000000	00000202	00000202
11	20202808	10101404	0	00000000	00000202	00000202
12	40405010	20202808	0	00000000	00000202	00000202
13	8080A020	40405010	0	00000000	00000202	00000202

c	$L(2^c)$	Shifted by 1	b	Placed	Row before	Row after
14	01014041	0080A020	0	00000000	00000202	00000202
15	02028082	01014041	1	00008000	00000202	00008202
16	04050104	02028082	0	00000000	00008202	00008202
17	080A0208	04050104	0	00000000	00008202	00008202
18	10140410	080A0208	0	00000000	00008202	00008202
19	20280820	10140410	0	00000000	00008202	00008202
20	40501040	20280820	0	00000000	00008202	00008202
21	80A02080	40501040	0	00000000	00008202	00008202
22	01404101	00A02080	0	00000000	00008202	00008202
23	02808202	01404101	1	00800000	00008202	00808202
24	05010404	02808202	0	00000000	00808202	00808202
25	0A020808	05010404	0	00000000	00808202	00808202
26	14041010	0A020808	0	00000000	00808202	00808202
27	28082020	14041010	0	00000000	00808202	00808202
28	50104040	28082020	0	00000000	00808202	00808202
29	A0208080	50104040	0	00000000	00808202	00808202
30	40410101	20208080	0	00000000	00808202	00808202
31	80820202	40410101	1	80000000	00808202	80808202

After the last column, the result is `80808202`. Its set positions are 1, 9, 15, 23 and 31. These are row coefficients, not the set positions in a single column $L(2^c)$; rotations wrap around the word. Attach the identity:

$$\begin{aligned}\texttt{identity_bit} &= 1 \ll (32+1) = \texttt{0000000200000000},\\ \texttt{augmented_row} &= \texttt{0000000080808202}\,|\,\texttt{0000000200000000}\\ &= \boxed{\texttt{0000000280808202}}.\end{aligned}$$

The high 32 bits are `00000002`, identity row 1; the low 32 bits are `80808202`, matrix row 1. This is a complete SM4 row, ready to append to `rows`. The other rows use their own row indices but exactly the same rule.

3.5 Step 3: pivots and XOR elimination

For each column, `pivot = None` means that no suitable row has been found yet. The search examines rows from the diagonal downward and stops at the first 1. No pivot means a singular matrix; the current implementation raises `StopIteration`. Correct SM4 L is invertible, so this is not a normal result.

The tuple assignment swaps the pivot into its diagonal position. The second loop clears that column in every other row:

```
rows[column], rows[pivot] = rows[pivot], rows[column]
rows[row] ^= rows[column]
```

Why XOR? Because $1 \oplus 1 = 0$, exactly the cancellation required over GF(2). The pivot is already 1, so division is unnecessary.

Here is the same method on a small 3×3 example, with conventional matrix column order. This matrix is only an illustration, not a reduced SM4 cipher:

$$\left[\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 0\\ 0 & 1 & 1 & 0 & 1 & 0\\ 1 & 0 & 0 & 0 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 \oplus R_0} \left[\begin{array}{ccc|ccc} 1 & 1 & 0 & 1 & 0 & 0\\ 0 & 1 & 1 & 0 & 1 & 0\\ 0 & 1 & 0 & 1 & 0 & 1 \end{array}\right].$$

Use row 1 to clear column 1 in rows 0 and 2:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 1 & 1 & 1 & 0\\ 0 & 1 & 1 & 0 & 1 & 0\\ 0 & 0 & 1 & 1 & 1 & 1 \end{array}\right].$$

Use row 2 to clear column 2 in rows 0 and 1:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{array}\right] = [I \mid M^{-1}].$$

For instance, this inverse maps $(1, 1, 1)^T$ to $(1, 0, 1)^T$. The real code performs the same operations for 32 columns.

3.6 Step 4: extract the inverse rows

```
inverse_row = augmented_row >> 32
inverse_matrix_rows.append(inverse_row)
return tuple(inverse_matrix_rows)
```

The lower half is now the identity. Shifting by 32 discards it and keeps the inverse row from the upper half. The tuple contains 32 integers, each encoding one row of M^{-1}.

4 `inverse_linear(value)`: apply the inverse

4.1 The distinction from `inverse_rows`

`inverse_rows()` prepares the transformation. `inverse_linear(value)` applies it to one observed 32-bit difference. The latter calls the former to obtain its cached matrix, so callers do not need to prepare it manually.

4.2 Follow one output bit

```
selected_bits = row & value
set_bit_count = bin(selected_bits).count("1")
parity = set_bit_count & 1
result |= parity << index
```

AND selects the participating input bits. Counting ones and keeping the low count bit computes their parity: odd count gives 1, even count gives 0. That is the GF(2) dot product of a matrix row and the input vector. For `value = 734E3D3D`, inverse row 0 gives

$$\texttt{51154505} \,\&\, \texttt{734E3D3D} = \texttt{51040505}.$$

Eight selected bits are 1, so output bit 0 is $8 \bmod 2 = 0$. Each of the 32 rows calculates one output bit. Shifting the parity positions that bit, and OR assembles distinct positions without disturbing previous bits. The complete result is

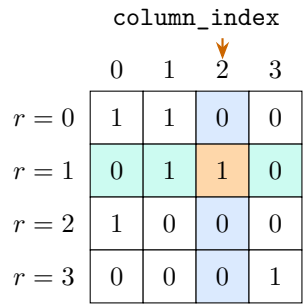
$$\texttt{inverse_linear(0x734E3D3D)} = \texttt{0x4F000000}.$$

Applying forward L returns `734E3D3D`, which checks the inversion.

5 Visual walkthrough: follow the loops through the matrix

The drawings use a four-bit teaching matrix so every cell remains visible. The real function uses the same indexing and operations with 32 bits. The small matrix below is invertible, but it is not the SM4 linear transformation.

5.1 First understand the two directions



Teal row	The outer loop fixes `row_index = 1`.
Blue column	The inner loop currently uses `column_index = 2`.
Orange cell	We read $M[1, 2] = 1$.

```
for row_index in range(4):        # move DOWN to the next row
    matrix_row = 0
    for column_index in range(4): # move RIGHT across this row
        output_bit = (columns[column_index] >> row_index) & 1
        matrix_row |= output_bit << column_index
```

The actual file uses `range(32)`. While the inner loop runs, `row_index` does not change. The visited cells are

$$(0,0) \to (0,1) \to (0,2) \to (0,3), \quad (1,0) \to (1,1) \to (1,2) \to (1,3), \quad \ldots$$

The arrays already contain columns; these loops collect their bits into rows. They do not move or modify the original matrix entries.

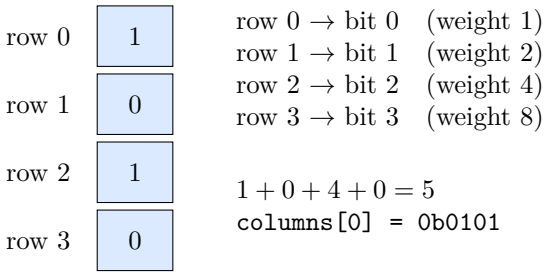
5.2 Matrix order and printed binary order are different

Matrix columns are drawn left-to-right as $c = 0, 1, 2, 3$. A printed binary integer is written with its highest bit first: $b_3 b_2 b_1 b_0$. The highlighted matrix row is $[0, 1, 1, 0]$. Its packed value is $0 \cdot 2^0 + 1 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 = 6$, printed as `0110`. For row 0, $[1, 1, 0, 0]$ is printed as `0011`, not `1100`.

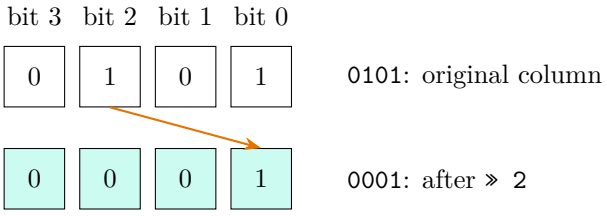
A shift operates on the stored integer. It does not slide a matrix cell visually to the right. The bit index tells us which coefficient to extract.

5.3 One column is an integer: why shift and AND?

Column 0 of the teaching matrix contains these entries from top to bottom:



To read row 2 from this column, shift its bit into position 0, then keep only that position. Every number below is a four-bit binary word:



The selected bit reaches position 0.
AND with `0001` discards every other bit.

$$(\texttt{0101} \gg 2)\ \&\ \texttt{0001} = \texttt{0001}\ \&\ \texttt{0001} = 1.$$

For row 1, instead:

$$(\texttt{0101} \gg 1)\ \&\ \texttt{0001} = \texttt{0010}\ \&\ \texttt{0001} = 0.$$

The other surviving 1 in `0010` does not matter: AND with 1 tests only the least significant bit. This is why shifting alone is insufficient.

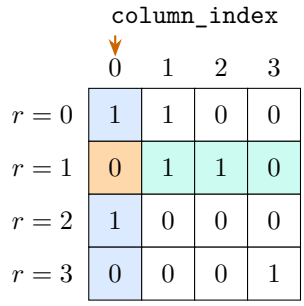
The four stored columns of this matrix are:

Index	0	1	2	3
Top-to-bottom entries	$[1, 0, 1, 0]$	$[1, 1, 0, 0]$	$[0, 1, 0, 0]$	$[0, 0, 0, 1]$
Stored binary	`0101`	`0011`	`0010`	`1000`

5.4 Four iterations: build row 1 from left to right

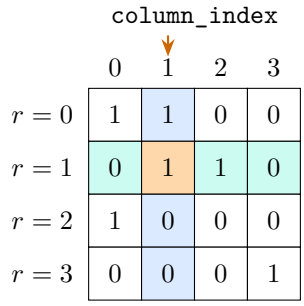
Keep `row_index = 1`. Start with `matrix_row = 0000`. Each orange cell is the next bit read; all binary words below have four bits.

Iteration 1: column 0



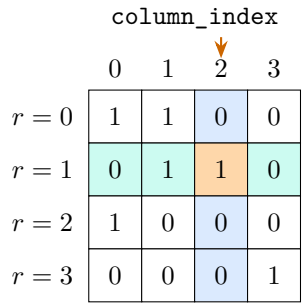
Read $(\texttt{0101} \gg 1) \,\&\, 1 = 0$.
Position it: $0 \ll 0 = \texttt{0000}$.
Accumulate: $\texttt{0000} \,|\, \texttt{0000} = \texttt{0000}$.

Iteration 2: column 1



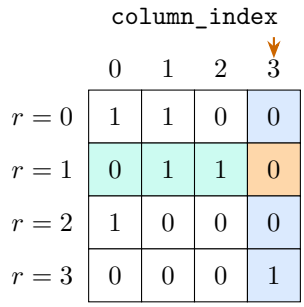
Read $(\texttt{0011} \gg 1) \,\&\, 1 = 1$.
Position it: $1 \ll 1 = \texttt{0010}$.
Accumulate: $\texttt{0000} \,|\, \texttt{0010} = \texttt{0010}$.

Iteration 3: column 2



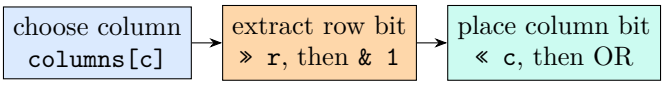
Read $(\texttt{0010} \gg 1) \,\&\, 1 = 1$.
Position it: $1 \ll 2 = \texttt{0100}$.
Accumulate: $\texttt{0010} \,|\, \texttt{0100} = \texttt{0110}$.

Iteration 4: column 3



Read $(\texttt{1000} \gg 1) \,\&\, 1 = 0$.
Position it: $0 \ll 3 = \texttt{0000}$.
Accumulate: $\texttt{0110} \,|\, \texttt{0000} = \texttt{0110}$.

The inner loop has reconstructed row 1: matrix entries $[0, 1, 1, 0]$, stored as integer 6, binary `0110`. OR accumulates set bits; OR with zero preserves the bits already written. Only after the inner loop finishes does the outer loop move down to row 2 and reset `matrix_row = 0`.



5.5 Where does the identity go in the packed row?

With four bits per half, the code's `32 + row_index` becomes `4 + row_index`. For row 1, the identity's 1 belongs at position 5:

$$\texttt{identity_bit} = 1 \ll (4 + 1) = \texttt{00100000}.$$

7	6	5	4	3	2	1	0
0	0	1	0	0	1	1	0

high half: row of I low half: row of M

$$\underbrace{\texttt{00000110}}_{\text{matrix row}} \mid \underbrace{\texttt{00100000}}_{\text{identity bit}} = \underbrace{\texttt{00100110}}_{\text{packed augmented row}}.$$

Mathematics writes $[M \mid I]$, but the integer above is printed high-to-low as $[I \mid M]$. This is a storage convention, not a different algorithm. For 32-bit SM4 halves the same structure uses 64 bit positions.

5.6 The elimination loop walks differently

The `column` loop now chooses a pivot column, rather than collecting a row. `candidate_row` searches down that column from the diagonal. After a pivot is found, `row` scans all other rows to eliminate their 1s.

R_0	1	1	0	0	pivot row: keep it
R_1	0	1	1	0	column 0 is 0: skip
R_2	1	0	0	0	column 0 is 1: XOR pivot
R_3	0	0	0	1	column 0 is 0: skip

Here $R_2 \leftarrow R_2 \oplus R_0$. Its matrix entries change from $[1, 0, 0, 0]$ to $[0, 1, 0, 0]$. Stored as integers, the XOR is $\texttt{0001} \oplus \texttt{0011} = \texttt{0010}$. The identity half must change too:

$$\texttt{01000001} \oplus \texttt{00010011} = \texttt{01010010}.$$

After all pivot columns, the low half is identity and the high half is inverse. The final `» 4` (`» 32` in SM4) keeps the high half and discards the low.

5.7 Finally, walk through the inverse matrix with the input vector

For this teaching matrix, Gaussian elimination gives

$$M^{-1} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Let $y = [1, 1, 1, 0]^T$, stored as `0111`. Each loop iteration selects one inverse row and calculates a single output bit. This is a row–vector dot product over GF(2), not another matrix inversion.

0	0	1	0		1
1	0	1	0	×	1
1	1	1	0		1
0	0	0	1		0

Highlighted row 1:
$(1 \cdot 1) \oplus (0 \cdot 1) \oplus (1 \cdot 1) \oplus (0 \cdot 0) = 0$

Index	Packed row	Input	AND	Ones / parity	Running result
0	`0100`	`0111`	`0100`	1 / 1	`0001`
1	`0101`	`0111`	`0101`	2 / 0	`0001`
2	`0111`	`0111`	`0111`	3 / 1	`0101`
3	`1000`	`0111`	`0000`	0 / 0	`0101`

The output is $x = [1, 0, 1, 0]^T$, stored as `0101`. Forward multiplication Mx restores $y = [1, 1, 1, 0]^T$, so the inverse result is checked.

Loop / operation	Picture to remember
`bit_index`	Feed one unit vector; obtain one column.
`row_index`, `column_index`	Fix a row; scan its cells left-to-right.
`» row_index`, then `& 1`	Bring one column bit to position 0; isolate it.
`« column_index`, then OR	Place that bit in the packed output row.
`column`, `candidate_row`	Choose a pivot column; search downward.
`rows[row] ^= rows[column]`	XOR the entire augmented row, both halves.
`augmented_row » 32`	Keep the inverse half after elimination.
`index, row` in `inverse_linear`	Select one inverse row; calculate one output bit.

In the real SM4 example, the same procedure gives L^{-1}(`734E3D3D`) = `4F000000`. The four-bit drawings illustrate storage and traversal; they do not replace the real 32-bit map.

6 `sm4_byte_candidates()`: solve one observation

6.1 What the function receives

Inputs are clean ciphertext, faulty ciphertext, selected byte and known mask. Neither the target key nor its internal trace is an argument. After validating lengths and ranges, `struct.unpack(">4I", ...)` splits each output into words. SM4 ciphertext order is $(X_{35}, X_{34}, X_{33}, X_{32})$.

6.2 Check the fault shape and undo L

The check `c[1:] != f[1:]` rejects changes in the other three words: the isolated last-round model requires them unchanged. The function then computes

$$\Delta X = c[0] \oplus f[0], \qquad \Delta B = L^{-1}(\Delta X).$$

For the first public-vector observation:

$$\texttt{681EDF34} \oplus \texttt{1B50E209} = \texttt{734E3D3D}, \qquad \Delta B = \texttt{4F000000}.$$

These shape checks are model-consistency checks, not proof of a physical single-bit error at the intended instruction.

6.3 Select the byte and observable constant

```
shift = 24 - 8 * byte
d = (delta_b >> shift) & 255
known = ((c[1] ^ c[2] ^ c[3]) >> shift) & 255
```

For byte 0, `shift = 24`, so $d =$ `4F`. The preceding condition rejects zero differences and any bits outside the selected byte. In `delta_b & ~(255 << shift)`, complement creates a mask for all other positions; AND detects unwanted differences there. Python's unbounded signed complement is harmless for this test with a nonnegative 32-bit difference.

The known data-word XOR is

$$\texttt{D206965E} \oplus \texttt{86B3E94F} \oplus \texttt{536E4246} = \texttt{07DB3D57},$$

so the first byte's observable constant is $q =$ `07`, stored as `known`. This XOR is the combination of the three newest input words, not the older word canceled when forming the ciphertext difference.

6.4 Try inputs, then convert them into key bytes

```
faulty_input = candidate_input ^ mask
output_difference = sm4.SBOX[candidate_input] ^ sm4.SBOX[faulty_input]
```

The loop visits 0–255 and keeps inputs satisfying $S(a) \oplus S(a \oplus e) = d$. With $e =$ `01` and $d =$ `4F`, the input guesses are `96` and `97`:

$$S(\texttt{96}) \oplus S(\texttt{97}) = \texttt{1A} \oplus \texttt{55} = \texttt{4F}.$$

Because $a = q \oplus k$, the next loop computes `key_byte = candidate_input ^ known`:

$$\texttt{96} \oplus \texttt{07} = \texttt{91}, \qquad \texttt{97} \oplus \texttt{07} = \texttt{90}.$$

The return tuple contains key candidates, input candidates, and the full `delta_b` word. One observation leaves ambiguity; this function does not select the reference key from that ambiguity.

7 `reverse_sm4()`: recover the master key

7.1 Why the input contains four subkeys

The function requires four consecutive final subkeys in ascending order: $rk_{28}, rk_{29}, rk_{30}, rk_{31}$. They correspond to schedule words $K_{32}, K_{33}, K_{34}, K_{35}$.

```
words = [None] * 32 + list(final_keys)
```

This creates 36 slots: 32 unknown words followed by four known seed words. One 32-bit final subkey is insufficient for this direct recurrence inversion.

7.2 Reverse one recurrence, then repeat

```
value = words[index+1] ^ words[index+2] ^ words[index+3] ^ sm4.CK[index]
words[index] = words[index+4] ^ sm4.linear_key(sm4.sbox(value))
```

The loop `range(31, -1, -1)` visits 31 down to 0. At each step the four words to the right are already known. It implements

$$K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i).$$

Here $T' = L' \circ \tau$, where $L'(z) = z \oplus \mathrm{ROL}_{13}(z) \oplus \mathrm{ROL}_{23}(z)$. This is different from data diffusion L. The constants CK_i are public: byte j is $7(4i + j) \bmod 256$, with bytes concatenated big-endian. For the first backward step ($i = 31$), substitute the recovered words:

$$\begin{aligned} Q &= \texttt{428D3654} \oplus \texttt{62293496} \oplus \texttt{01CF72E5} \oplus \texttt{646B7279} \\ &= \texttt{4500025E}, \\ \tau(Q) &= \texttt{73D6E99D}, \qquad T'(Q) = \texttt{605CAC93}, \\ K_{31} &= \texttt{9124A012} \oplus \texttt{605CAC93} = \texttt{F1780C81}. \end{aligned}$$

The same rule repeats down to index 0; no further faulty encryptions are needed for this inversion. The optional observer receives the newly calculated index and word for teaching. It is not required to calculate the key.

7.3 Remove FK and serialize

```
master_word = words[word_index] ^ sm4.FK[word_index]
return struct.pack(">4I", *master_words)
```

The initial words were whitened by fixed public constants, so XOR removes them. The four results are serialized into the 16-byte master key. For word 0: `A292FFA1` $\oplus$ `A3B1BAC6` = `01234567`. The star expands the four-word list into four arguments to `struct.pack`.

8 `peel_sm4()`: expose an earlier round

8.1 Restore chronological order

This function receives a ciphertext, a target round in 28–31 and a dictionary containing only already recovered keys. It validates the block, unpacks its four words and reverses their order:

```
words = list(struct.unpack(">4I", ciphertext))[::-1]
```

Ciphertext order is reversed relative to the rolling cipher state. For the public vector the chronological window is

$$[\texttt{536E4246}, \texttt{86B3E94F}, \texttt{D206965E}, \texttt{681EDF34}].$$

8.2 Undo only rounds with recovered keys

For target round 31, `range(31, 31, -1)` is empty: no round is peeled. For target round 30, only round 31 is undone. If its key is missing, the function raises `ValueError` instead of guessing a key.

```
previous = words[3] ^ sm4.linear(sm4.sbox(words[0] ^ words[1] ^ words[2] ^
↪  recovered_keys[number]))
words = [previous, words[0], words[1], words[2]]
```

This solves the forward recurrence for the state word that was dropped. Using recovered $rk_{31} =$ `9124A012`:

$$A = \texttt{536E4246} \oplus \texttt{86B3E94F} \oplus \texttt{D206965E} \oplus \texttt{9124A012} = \texttt{96FF9D45},$$

$$\tau(A) = \texttt{1A488C73}, \quad L(\tau(A)) = \texttt{138D5078}, \quad X_{31} = \texttt{681EDF34} \oplus \texttt{138D5078} = \texttt{7B938F4C}.$$

The new chronological window is

$$[\texttt{7B938F4C}, \texttt{536E4246}, \texttt{86B3E94F}, \texttt{D206965E}].$$

Finally the function reverses and packs it again, preserving the ciphertext-like interface expected by the byte filter. The same peeling is applied separately to correct and faulty outputs; teacher trace values are not used.

9 `recover_sm4_master_key()`: coordinate the attack

9.1 Inputs and validation

`correct` is the clean ciphertext. `fault_oracle(round, byte, mask)` asks the target for a faulty ciphertext. `masks` is converted to a tuple, then checked for a nonempty sequence, distinct elements and values 1–255. `start_byte` must be 0–3. Repeating the same mask on the same deterministic input does not supply a new constraint, which motivates the duplicate check.

`candidate_filter=None` selects the known-mask filter described above. A supplied filter can replace that equation, as in the Giraud-inspired adapter. Its affected byte must match the requested byte. An optional `observer` receives copies of event dictionaries for presentation.

9.2 Initialize state and choose byte order

`keys` stores uniquely recovered round keys; `history` stores observation records; `counts` stores the latest surviving-candidate count for each byte; `used` counts faulty outputs. The outer loop visits rounds 31, 30, 29 and 28. Each begins with a clean view obtained by peeling only later rounds whose subkeys are in `keys`. `bytearray(4)` allocates the four bytes of the current subkey.

```
for offset in range(4):
    byte_order.append((start_byte + offset) % 4)
```

For `start_byte=2`, this gives $(2+0) \bmod 4 = 2$, $(2+1) \bmod 4 = 3$, $(2+2) \bmod 4 = 0$, $(2+3) \bmod 4 = 1$: order $[2, 3, 0, 1]$. This changes the visiting order, not byte positions in the assembled key.

9.3 Ask for faults and intersect candidates

For each byte, `survivors=None` means no observation yet; it is not an empty candidate set. `enumerate(masks, 1)` provides observation numbers starting at 1 and their masks. The oracle returns a faulty ciphertext, it is peeled, and the byte filter computes a fresh candidate set.

```
if survivors is None:
    before = 256
    survivors = set(fresh)
else:
    before = len(survivors)
    survivors = survivors & fresh
```

With the first-byte example:

$$\{90, 91\} \cap \{70, 72, 93, 91\} = \{91\}.$$

The second set comes from mask 02 and difference 81. Intersection can remove candidates but never introduce new ones. An empty intersection raises an error because the observations are incompatible with the model.

9.4 Record evidence; accept only uniqueness

After each valid observation, `used` increases and `counts` is updated. The event dictionary records round, byte, mask, counts, surviving values, S-box inputs, inverse difference, ciphertexts, peeled views and the keys used for peeling. It supports debugging and teaching; it is not an additional source of secret key information.

```
if len(survivors) == 1:
    subkey[byte] = next(iter(survivors))
    break
```

The set has exactly one element, so extracting that element is unambiguous. `break` stops requesting faults for this byte. It does not exit the round loop. The `else` attached to the observation `for` executes only if its mask budget is exhausted without that break. The function then returns `complete=False`, `master_key=None` and the blocked round; it never pretends a guessed byte is recovered.

9.5 Assemble subkeys, then reverse the schedule

```
keys[number] = int.from_bytes(subkey, "big")
```

For bytes `91 24 A0 12`, this yields integer `9124A012`. Only after all four bytes are unique is the subkey available for peeling. The final loop builds `final_keys` in ascending order 28–31, even though the attack recovered them in descending order. It then calls `reverse_sm4(final_keys)` and returns the master key, four subkeys, history, counts, query cost and `complete=True`.

Recovered subkey	Value	Cumulative faulty outputs
rk_{31}	9124A012	8
rk_{30}	01CF72E5	16
rk_{29}	62293496	24
rk_{28}	428D3654	32

This default-vector run uses 32 faulty outputs, plus the clean reference. It recovers $MK =$ `0123456789ABCDEFFEDCBA9876543210`. The observation count is an example result, not a universal physical requirement.

10 `main()`: connect the terminal interface

`ArgumentParser` reads command-line settings. The algorithm option is fixed to SM4. `-step` enables terminal lesson pauses; `-color` controls ANSI color; `-faults` limits observations per byte;

`-row` chooses the first byte to demonstrate. Its name is retained for interface compatibility, not because SM4 uses an AES-style matrix.

`-column` and `-seed` are accepted for compatibility but are not forwarded to `run.sm4_demo` by this entry point. They do not change this deterministic SM4 scenario. The explicit fault-limit check rejects values outside 1–255.

The local `import run` defers loading presentation until terminal execution is requested. The call

```
run.sm4_demo(run.LessonTerminal(args.color, args.step), args.faults, args.row)
```

connects settings to the existing shared lesson. Successful recovery returns exit code 0; incomplete recovery returns 2.

10.1 The final entry-point guard and logs

```
if __name__ == "__main__":
    from terminal_log import run_with_log
    raise SystemExit(run_with_log(main, "piret_sm4"))
```

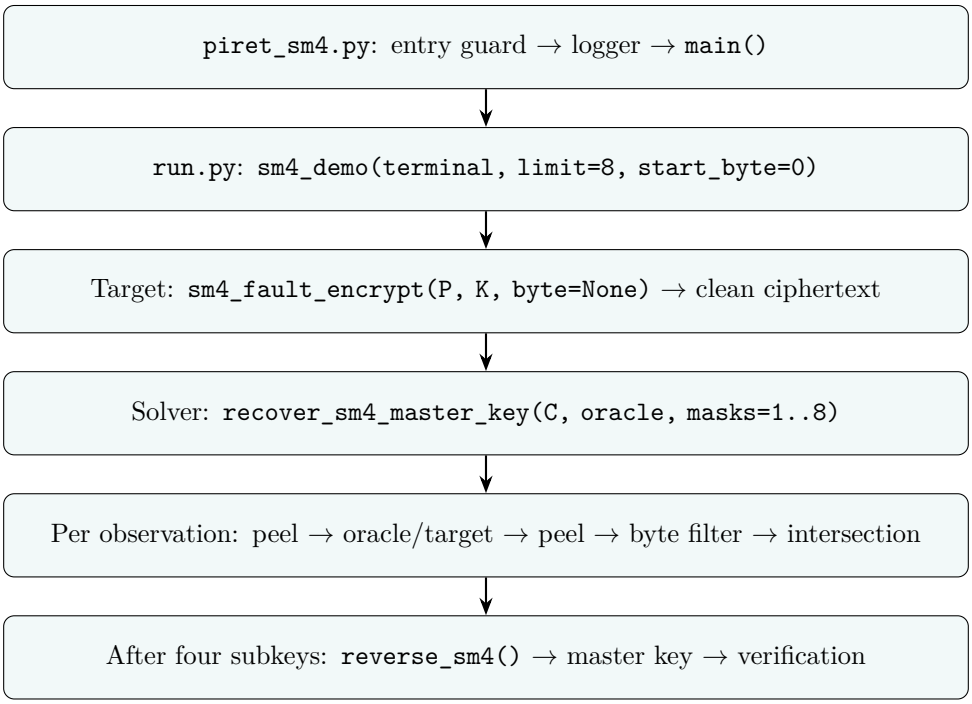
This block runs when the file is executed directly. Importing the functions does not launch a lesson. The wrapper captures terminal execution in `logs/piret_sm4.txt` and colored HTML; files are overwritten per run. `SystemExit` passes the resulting status to the shell.

11 Follow one real execution from the command to the recovered key

This section is an execution route, not another independent description of the functions. Read the numbered stages in order: the output of one call becomes an input to the next. Numerical records are generated from the current solver. The builder also runs the real terminal entry point without interactive pauses and verifies that it succeeds. The command below adds pauses but follows the same mathematical path:

```
python src/piret_sm4.py --faults 8 --row 0 --step
```

11.1 The route through the files



The target knows the secret key. The solver receives ciphertexts and fault descriptors. The terminal's teaching callbacks display additional internals; their displays are not secret inputs to the candidate equations.

11.2 Stage 1: argument parsing and the clean target call

The entry guard calls `run_with_log(main, "piret_sm4")`. The parser obtains `faults=8`, `row=0`, `step=True`, with automatic color mode. `main()` constructs `LessonTerminal` and invokes `run.sm4_demo`. That function converts `sm4.KEY` and `sm4.PLAINTEXT` from hexadecimal strings to bytes. For this public demonstration vector:

$$P = K = \texttt{0123456789ABCDEFFEDCBA9876543210}.$$

The clean call uses `byte=None`; the observer records events without injecting any fault. The result is

$$C = \texttt{681EDF34D206965E86B3E94F536E4246}.$$

The teacher can display the clean round-31 trace $A = \texttt{96FF9D45}$, $B = \texttt{1A488C73}$, $L(B) = \texttt{138D5078}$. These values explain the target; the solver does not receive this trace.

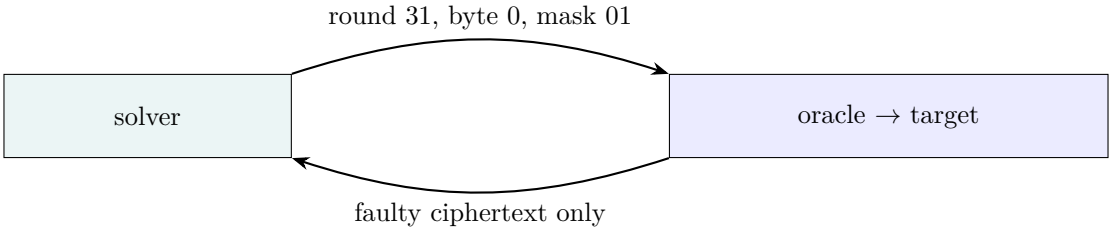
11.3 Stage 2: enter the solver and select the first byte

The actual call passes `range(1, 9)` as permitted masks and the two callbacks `oracle` and `report`. Solver state starts as

```
keys = {}
history = []
counts = {}
used = 0
```

The outer round loop selects `number=31`. The call `peel_sm4(C, 31, {})` undoes no rounds and returns C unchanged. The subkey buffer is `00 00 00 00`; `start_byte=0` produces visiting order $[0, 1, 2, 3]$. The first selected byte is 0, `survivors=None`, and `enumerate(masks, 1)` produces observation 1, mask 01.

11.4 Stage 3: the oracle enters the target, then returns only ciphertext



`oracle(31, 0, 1)` calls `sm4_fault_encrypt(P, K, byte=0, mask=1, fault_round=31)`. The callback intervenes only at the selected S-box input:

$$\begin{aligned} \text{shift} &= 24 - 8(0) = 24, \\ \text{shifted mask} &= \texttt{01} \ll 24 = \texttt{01000000}, \\ A' &= \texttt{96FF9D45} \oplus \texttt{01000000} = \texttt{97FF9D45}. \end{aligned}$$

After the target completes normal SM4 processing, the oracle returns

$$F_1 = \texttt{1B50E209D206965E86B3E94F536E4246}.$$

The terminal may illustrate fault propagation using its teacher trace. The solver gets F_1 alone. It calls `peel_sm4(F1,31,keys)`, which also undoes no rounds.

11.5 Stage 4: follow the first byte-filter call

`sm4_byte_candidates(C,F1,byte=0,mask=1)` unpacks four big-endian words and checks that the last three words match. It computes

$$\Delta X = \texttt{681EDF34} \oplus \texttt{1B50E209} = \texttt{734E3D3D}.$$

Its call to `inverse_linear(0x734E3D3D)` obtains the fixed inverse rows. On a fresh process, `inverse_rows()` builds and caches them; subsequent observations reuse the matrix. The earlier illustrated matrix sections explain that preparation. Applying its rows gives

$$\Delta B = \texttt{4F000000}, \qquad d = (\texttt{4F000000} \gg 24)\ \&\ \texttt{FF} = \texttt{4F}.$$

The other three words yield the public constant

$$q_{\text{word}} = \texttt{D206965E} \oplus \texttt{86B3E94F} \oplus \texttt{536E4246} = \texttt{07DB3D57}, \qquad q = \texttt{07}.$$

The 256-input loop retains only $a = \texttt{96}$ and $a = \texttt{97}$ because $S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}$. Converting input guesses to key guesses gives $\texttt{96} \oplus \texttt{07} = \texttt{91}$ and $\texttt{97} \oplus \texttt{07} = \texttt{90}$. The function returns

```
fresh  = {0x90, 0x91}
inputs = {0x96, 0x97}
delta  = 0x4F000000
```

Back in the solver, `survivors=None` selects the first-observation branch: `before=256` and `survivors=set(fresh)`. `used` becomes 1 and `counts["31:0"]` becomes 2. An event is appended and passed to `report(event)` for display. Since two candidates remain, the solver does not write a subkey byte yet.

11.6 Stage 5: the second observation resolves the first byte

The next observation requests `oracle(31,0,2)`. It returns

$$F_2 = \texttt{ED9DD932D206965E86B3E94F536E4246}.$$

The same filter computes

$$\Delta X = \texttt{681EDF34} \oplus \texttt{ED9DD932} = \texttt{85830606},$$
$$\Delta B = L^{-1}(\texttt{85830606}) = \texttt{81000000}, \qquad d = \texttt{81}.$$

Its compatible S-box inputs are $\{\texttt{75}, \texttt{77}, \texttt{94}, \texttt{96}\}$, giving fresh key guesses $\{\texttt{70}, \texttt{72}, \texttt{93}, \texttt{91}\}$. The existing survivor set is now intersected:

$$\{\texttt{90}, \texttt{91}\} \cap \{\texttt{70}, \texttt{72}, \texttt{93}, \texttt{91}\} = \{\texttt{91}\}.$$

Now `used=2`, `counts["31:0"]=1` and `subkey[0]=0x91`. The buffer becomes `91 00 00 00`; the other positions are not recovered yet. The `break` exits the mask loop, so mask 03 is not requested for this byte.

11.7 Stages 6–9: follow all observations and assemble each subkey

The next tables are generated from the real event history, in execution order. They connect loop variables, inverse differences, candidate intersections, byte-buffer updates and subkey milestones. All hexadecimal entries denote values, not variable names. Query numbering excludes the clean reference.

Round 31: input views and the byte loop

Before this round, recovered subkeys are: `none`. The clean view returned by `peel_sm4` is

$$C_{\text{view}} = \texttt{681EDF34D206965E86B3E94F536E4246}.$$

The first oracle response for this round and its peeled view are

$$F = \texttt{1B50E209D206965E86B3E94F536E4246},$$
$$F_{\text{view}} = \texttt{1B50E209D206965E86B3E94F536E4246}.$$

The filter operates on these views. In the table, "fresh" is the key-byte set for this pair and "remaining" is the intersection with earlier observations for the same byte.

Query	Byte	Mask	ΔB	Fresh	Remaining
1	0	01	4F000000	90 91	90 91
2	0	02	81000000	70 72 91 93	91
3	1	01	00710000	24 25	24 25
4	1	02	00830000	24 26	24
5	2	01	00007900	A0 A1	A0 A1
6	2	02	00006F00	A0 A2	A0
7	3	01	00000080	12 13	12 13
8	3	02	000000C9	10 12	12

Unique bytes are written into the subkey buffer in the following order. Zero-filled positions not yet written are placeholders, not recovered bytes.

Byte written	Value	Buffer after write	Queries used
0	91	91000000	2
1	24	91240000	4
2	A0	9124A000	6
3	12	9124A012	8

Only now is `keys[31]` assigned: rk_{31} = `9124A012`. This word may be used to peel the next attacked round.

Round 30: input views and the byte loop

Before this round, recovered subkeys are: `rk31=9124A012`. The clean view returned by `peel_sm4` is

$$C_{\text{view}} = \texttt{D206965E86B3E94F536E42467B938F4C}.$$

For the clean ciphertext, the peeling calls perform the following actual calculations in order. The last column is the recovered older state word:

Undone round	S-box input	τ output	L output	Older word
31	96FF9D45	1A488C73	138D5078	7B938F4C

The first oracle response for this round and its peeled view are

$$F = \texttt{AE21A70A772712DA86B3E94F536E4246},$$
$$F_{\text{view}} = \texttt{772712DA86B3E94F536E42467B938F4C}.$$

The filter operates on these views. In the table, "fresh" is the key-byte set for this pair and "remaining" is the intersection with earlier observations for the same byte.

Query	Byte	Mask	ΔB	Fresh	Remaining
9	0	01	21000000	00 01	00 01
10	0	02	3C000000	01 03	01
11	1	01	00550000	CE CF	CE CF
12	1	02	006D0000	CD CF	CF
13	2	01	00005100	72 73	72 73
14	2	02	0000AB00	70 72	72
15	3	01	000000EB	E4 E5	E4 E5
16	3	02	000000FF	E5 E7	E5

Unique bytes are written into the subkey buffer in the following order. Zero-filled positions not yet written are placeholders, not recovered bytes.

Byte written	Value	Buffer after write	Queries used
0	01	01000000	10
1	CF	01CF0000	12
2	72	01CF7200	14
3	E5	01CF72E5	16

Only now is `keys[30]` assigned: rk_{30} = `01CF72E5`. This word may be used to peel the next attacked round.

Round 29: input views and the byte loop

Before this round, recovered subkeys are: `rk31=9124A012, rk30=01CF72E5`. The clean view returned by `peel_sm4` is

$$C_{\text{view}} = \texttt{86B3E94F536E42467B938F4C893450AD}.$$

For the clean ciphertext, the peeling calls perform the following actual calculations in order. The last column is the recovered older state word:

Undone round	S-box input	τ output	L output	Older word
31	96FF9D45	1A488C73	138D5078	7B938F4C
30	AF8156A0	6FBFDA1D	5B32C6F3	893450AD

The first oracle response for this round and its peeled view are

$$F = \texttt{FD15CC075271D40B4F8E1DBB536E4246},$$
$$F_{\text{view}} = \texttt{4F8E1DBB536E42467B938F4C893450AD}.$$

The filter operates on these views. In the table, "fresh" is the key-byte set for this pair and "remaining" is the intersection with earlier observations for the same byte.

Query	Byte	Mask	ΔB	Fresh	Remaining
17	0	01	3D000000	62 63	62 63
18	0	02	89000000	60 62	62
19	1	01	00E00000	28 29	28 29
20	1	02	001E0000	29 2B	29
21	2	01	0000E900	34 35	34 35
22	2	02	00008200	34 36	34
23	3	01	00000057	96 97	96 97
24	3	02	0000001A	94 96	96

Unique bytes are written into the subkey buffer in the following order. Zero-filled positions not yet written are placeholders, not recovered bytes.

Byte written	Value	Buffer after write	Queries used
0	62	62000000	18
1	29	62290000	20
2	34	62293400	22
3	96	62293496	24

Only now is `keys[29]` assigned: $rk_{29} =$ `62293496`. This word may be used to peel the next attacked round.

Round 28: input views and the byte loop

Before this round, recovered subkeys are: `rk31=9124A012, rk30=01CF72E5, rk29=62293496`. The clean view returned by `peel_sm4` is

$$C_{\text{view}} = \texttt{536E42467B938F4C893450AD6FE46B75}.$$

For the clean ciphertext, the peeling calls perform the following actual calculations in order. The last column is the recovered older state word:

Undone round	S-box input	τ output	L output	Older word
31	96FF9D45	1A488C73	138D5078	7B938F4C
30	AF8156A0	6FBFDA1D	5B32C6F3	893450AD
29	C3E0A931	928929B3	E957823A	6FE46B75

The first oracle response for this round and its peeled view are

$$F = \texttt{84EB762070DAF490AA919B05476A5256},$$
$$F_{\text{view}} = \texttt{476A52567B938F4C893450AD6FE46B75}.$$

The filter operates on these views. In the table, "fresh" is the key-byte set for this pair and "remaining" is the intersection with earlier observations for the same byte.

Query	Byte	Mask	ΔB	Fresh	Remaining
25	0	01	04000000	42 43	42 43
26	0	02	55000000	40 42	42
27	1	01	00820000	8C 8D	8C 8D
28	1	02	00450000	8D 8F	8D
29	2	01	00005800	36 37	36 37
30	2	02	00006000	34 36	36
31	3	01	00000096	54 55	54 55
32	3	02	00000022	54 56	54

Unique bytes are written into the subkey buffer in the following order. Zero-filled positions not yet written are placeholders, not recovered bytes.

Byte written	Value	Buffer after write	Queries used
0	42	42000000	26
1	8D	428D0000	28
2	36	428D3600	30
3	54	428D3654	32

Only now is `keys[28]` assigned: $rk_{28} = \texttt{428D3654}$. The four required subkeys are now known; leave the round loop and reverse the key schedule.

11.8 Stage 10: four recovered words enter reverse expansion

The attack's recovery order was 31, 30, 29, 28. The final list is deliberately assembled in the ascending order expected by `reverse_sm4`:

```
final_keys = [0x428D3654, 0x62293496, 0x01CF72E5, 0x9124A012]
```

These are $K_{32}, K_{33}, K_{34}, K_{35}$. The first backward calculation is

$$Q = \texttt{428D3654} \oplus \texttt{62293496} \oplus \texttt{01CF72E5} \oplus \texttt{646B7279} = \texttt{4500025E},$$
$$\tau(Q) = \texttt{73D6E99D}, \qquad T'(Q) = \texttt{605CAC93},$$
$$K_{31} = \texttt{9124A012} \oplus \texttt{605CAC93} = \texttt{F1780C81}.$$

Each following index reuses words already calculated to its right. No new oracle requests occur in this phase. At the end, remove the four FK words:

i	K_i	FK_i	$MK_i = K_i \oplus FK_i$
0	A292FFA1	A3B1BAC6	01234567
1	DF01FEBF	56AA3350	89ABCDEF
2	99A12B0F	677D9197	FEDCBA98
3	C42410CC	B27022DC	76543210

Concatenation returns the 16 bytes `0123456789ABCDEFFEDCBA9876543210`.

11.9 Stage 11: return values, verification and process exit

The solver returns `complete=True`, the recovered master key, four subkeys, 32 events, candidate counts and `used=32`. The terminal's schedule display must agree with that key. The verified round trip is

$$\text{Encrypt}(P, MK) = \texttt{681EDF34D206965E86B3E94F536E4246} = C,$$
$$\text{Decrypt}(C, MK) = \texttt{0123456789ABCDEFFEDCBA9876543210} = P.$$

The simulation also compares the recovered key with the target's reference key, only for validation. `sm4_demo` returns `True`; `main` returns 0; the logging wrapper finishes and `SystemExit` passes status 0 to the shell.

The 32 faulty ciphertexts are this software example's recovery cost. Teacher callbacks may recompute values for display; those calculations are not additional independent observations. Physical glitch calibration and failed acquisitions are not counted or demonstrated here.

12 One complete practical story

The target encrypts a public plaintext with its hidden key. The solver gets a clean ciphertext. It requests two useful fault observations for the first byte, undoes L, intersects candidates and obtains 91. It repeats for the remaining three bytes to obtain rk_{31}. Only then can it peel round 31 and attack round 30. The process continues through round 28. Four consecutive recovered subkeys seed reverse expansion, whose initial FK whitening is undone to obtain the master key. Re-encryption and decryption verify the result.

The source's known-mask solver and the physical injection mechanism are separate questions. A valid laboratory evaluation must establish whether real faults match the assumed round, location and magnitude; the exact XOR in the Python callback is a controlled teaching model.

12.1 Build this document

```
python build_piret_sm4_guide.py
```

12.2 Source and verification record

This edition covers all eight top-level functions and the nested observer in `src/piret_sm4.py`. Source SHA-256: 424717e680ab76021831a9a6eaa490161342f43b21d4c8d086297ee97c051032. Its generated examples were checked using an actual complete recovery, matrix/parity checks, candidate intersections, peeling, re-encryption and decryption.